Taller: sessionStorage

Objetivo del taller

Al terminar, comprobarás cómo una página guarda un dato en **sessionStorage**, cómo otra página lo lee y cuándo ese dato desaparece.

- **Qué vas a construir:** dos páginas. En la primera escribes tu nombre y lo guardas; la segunda te saluda con ese nombre y tiene un botón para borrarlo.
- **Qué vas a practicar:** `setItem`, `getItem` y `removeItem`, funciones separadas de los eventos, y la pestaña *Application* de las herramientas del navegador.

Necesitas:

- ☐ Visual Studio Code.
- ☐ Google Chrome o Microsoft Edge. Con ellos basta abrir los archivos con doble clic; no hace falta Live Server.

Atajos que usarás:

Acción	Atajo
Abrir las herramientas del navegador	F12
Recargar la página	F5
Guardar un archivo en VS Code	Ctrl + S

Conceptos clave

Las páginas web olvidan todo al cambiar de página; sessionStorage es la "bodega" del navegador que permite pasar un dato de una página a otra.

El problema: las páginas no tienen memoria. Cada vez que el navegador abre una página, la carga desde cero. Las variables de JavaScript de la página anterior se borran, así que la página 2 no tiene forma de saber qué escribiste en la página 1.

La solución: sessionStorage. Su nombre viene del inglés *session* (sesión) y *storage* (almacenamiento). Es un pequeño almacén del navegador que guarda datos **mientras la pestaña esté abierta**. Cualquier página del mismo sitio, en esa pestaña, puede leerlo.

Cómo guarda los datos: clave → valor. Funciona como un diccionario: cada dato tiene un nombre (la clave) y un contenido (el valor). En este taller guardaremos:

```
nombre  →  "Ana"
```

Los 3 métodos que vas a usar:

Método	Qué hace	Dónde lo usaremos
`sessionStorage.setItem("nombre", valor)`	Guarda un dato	Página 1, al presionar *Guardar*
`sessionStorage.getItem("nombre")`	Lee un dato. Si no existe, devuelve `null`	Página 2, al cargar
`sessionStorage.removeItem("nombre")`	Borra un dato	Página 2, al presionar *Borrar*

> ***Analogía:*** *es como la manilla de un evento. La página 1 te pone la manilla con tu nombre (`setItem`), la página 2 la revisa (`getItem`) y el botón Borrar te la quita (`removeItem`). Al cerrar la pestaña, la manilla se bota.*

Paso 1: Crear la carpeta y los archivos

El proyecto tiene cuatro archivos: dos páginas HTML y un archivo JavaScript para cada una.

```
ejemplo-sessionstorage/
├── pagina1.html
├── pagina2.html
└── JS/
    ├── pagina1.js
    └── pagina2.js
```

1. Crea en tu computador una carpeta llamada `ejemplo-sessionstorage`.
2. En VS Code, ábrela con **File > Open Folder**.
3. En el Explorador, crea los archivos `pagina1.html` y `pagina2.html` con el ícono **New File**.
4. Crea la carpeta `JS` con el ícono **New Folder** y, dentro de ella, los archivos `pagina1.js` y `pagina2.js`.

> *Escribe los nombres exactamente así, con `JS` en mayúsculas. Si en el HTML escribes `JS/pagina1.js` pero la carpeta se llama `js`, el navegador no encontrará el archivo cuando lo publiques en GitHub Pages.*

Paso 2: La página 1 guarda el nombre

La página 1 tiene una caja de texto y un botón. Al hacer clic, JavaScript guarda lo escrito en sessionStorage y abre la página 2.

`pagina1.html`

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Página 1</title>
    <script src="JS/pagina1.js" defer></script>
</head>
<body>
    <h1>Página 1</h1>
```

```
    <input type="text" id="nombre" placeholder="Escribe tu nombre">
    <button id="btn-guardar">Guardar e ir a la página 2</button>
</body>
</html>
```

- El `<script>` va en el `<head>` con **`defer`**: así el navegador ejecuta el JavaScript cuando ya leyó todo el HTML y existen la caja y el botón.
- Los `id` (`nombre` y `btn-guardar`) son los nombres con los que JavaScript encuentra cada elemento.

`JS/pagina1.js`

```
// Función: guarda el nombre y cambia de página
function guardar() {
    const nombre = document.getElementById("nombre").value;
    sessionStorage.setItem("nombre", nombre);   // guarda en la bodega
    window.location.href = "pagina2.html";      // cambia de página
}

// Evento: al hacer clic en el botón, se ejecuta guardar
document.getElementById("btn-guardar").addEventListener("click", guardar);
```

Línea por línea:

Línea	Qué hace
`function guardar() { ... }`	Declara una función llamada `guardar`. Todavía no se ejecuta: solo queda lista
`document.getElementById("nombre").value`	Busca la caja con `id="nombre"` y lee lo que el usuario escribió en ella
`const nombre = ...`	Guarda ese texto en una variable llamada `nombre`
`sessionStorage.setItem("nombre", nombre)`	Guarda en la bodega un dato con la clave `"nombre"` y el valor que escribió el usuario
`window.location.href = "pagina2.html"`	Cambia de página. Aquí se pierden todas las variables, pero lo que está en sessionStorage sobrevive
`addEventListener("click", guardar)`	Conecta el botón con la función: cuando hagan clic, el navegador ejecuta `guardar`

> ***Ojo:*** *en `addEventListener("click", guardar)` la función va **sin paréntesis**. Con paréntesis, `guardar()` se ejecutaría de inmediato al cargar la página, antes de que el usuario escriba nada.*
>
> ***Detalle:*** *la clave `"nombre"` (entre comillas) es la etiqueta en la bodega; la variable `nombre` (sin comillas) es el texto escrito. Se llaman igual para que sea fácil de leer, pero son cosas distintas.*

Guarda los dos archivos con **Ctrl + S**.

Paso 3: La página 2 lee y borra el nombre

La página 2 nunca vio la caja de texto de la página 1: solo sabe tu nombre porque lo lee de sessionStorage.

`pagina2.html`

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Página 2</title>
    <script src="JS/pagina2.js" defer></script>
</head>
<body>
    <h1>Página 2</h1>

    <p id="saludo"></p>
    <button id="btn-borrar">Borrar el nombre</button>
    <a href="pagina1.html">Volver a la página 1</a>
</body>
</html>
```

El párrafo `<p id="saludo">` empieza vacío: JavaScript lo llenará con el saludo.

`JS/pagina2.js`

```
// Función: lee el nombre y lo muestra
function mostrarNombre() {
    const nombre = sessionStorage.getItem("nombre");   // lee de la bodega

    if (nombre === null) {
        document.getElementById("saludo").textContent = "No hay ningún nombre
guardado.";
    } else {
        document.getElementById("saludo").textContent = "Hola, " + nombre;
    }
}

// Función: borra el nombre y vuelve a mostrar el resultado
function borrar() {
    sessionStorage.removeItem("nombre");   // borra de la bodega
    mostrarNombre();                       // actualiza el mensaje
}

// Eventos
document.addEventListener("DOMContentLoaded", mostrarNombre);            // al cargar
la página
document.getElementById("btn-borrar").addEventListener("click", borrar); // al hacer
clic
```

Línea por línea:

Línea	Qué hace
`sessionStorage.getItem("nombre")`	Busca en la bodega el dato con la clave `"nombre"`.

Línea	Qué hace
	Si existe, devuelve el texto; si no, devuelve `null`
`if (nombre === null)`	Pregunta si la bodega estaba vacía. `null` significa "no hay nada guardado"
`.textContent = "No hay ningún nombre guardado."`	Si no había nada, escribe ese mensaje en el párrafo `saludo`
`.textContent = "Hola, " + nombre`	Si había un nombre, escribe el saludo. El `+` une los dos textos
`sessionStorage.removeItem("nombre")`	Borra de la bodega el dato con la clave `"nombre"`
`mostrarNombre()`	Vuelve a ejecutar la función de mostrar. Como ya no hay dato, aparece el mensaje de vacío
`addEventListener("DOMContentLoaded", mostrarNombre)`	Ejecuta `mostrarNombre` en cuanto el navegador termina de leer el HTML
`addEventListener("click", borrar)`	Ejecuta `borrar` cuando hacen clic en el botón

> ***¿Qué es `DOMContentLoaded`?*** *Es el evento que el navegador dispara cuando terminó de leer todo el HTML de la página. Lo usamos para que `mostrarNombre` también se ejecute desde un evento, igual que `borrar`.*
>
> ***Funciones reutilizables:*** *`borrar` llama a `mostrarNombre()` para actualizar el mensaje sin recargar la página. Por eso conviene tener las funciones separadas de los eventos: la misma función se usa al cargar la página y después de borrar.*

Guarda los dos archivos con **Ctrl + S**.

Paso 4: Pruebas guiadas

Con seis pruebas comprobarás cuándo el dato se guarda, cuándo sobrevive y cuándo desaparece. Antes de empezar, prepara la vista de la bodega.

Preparar la vista de sessionStorage

1. Abre `pagina1.html` con doble clic. Se abre en el navegador.
2. Presiona **F12** y ve a la pestaña **Application** (en español, *Aplicación*). Si no la ves, búscala con la flecha **»**.
3. En el panel izquierdo, despliega **Session Storage** y haz clic en la dirección que aparece debajo. Ahí verás el contenido de la bodega. Por ahora está vacía.

Deja las herramientas abiertas durante todas las pruebas.

#	Qué haces	Qué debe pasar	Por qué
1	En la página 1, escribe tu nombre y presiona **Guardar e ir**	La página 2 dice *Hola, Ana*. En *Session Storage* aparece	`setItem` guardó el dato y `getItem` lo leyó desde otra

#	Qué haces	Qué debe pasar	Por qué
	a la página 2	`nombre` con tu nombre	página
2	En la página 2, presiona **F5**	El saludo sigue igual	Recargar no borra sessionStorage: es la misma pestaña
3	Presiona **Borrar el nombre**	Aparece *No hay ningún nombre guardado*. El dato desaparece de *Session Storage*	`removeItem` borró el dato y `getItem` devolvió `null`
4	Haz clic en **Volver a la página 1**, guarda otro nombre y luego **cierra la pestaña**	La pestaña se cierra; en la prueba 5 comprobarás si el dato sobrevivió	Al cerrar la pestaña, el navegador descarta su bodega
5	Abre `pagina2.html` con doble clic en una pestaña nueva	Aparece *No hay ningún nombre guardado*	Cada pestaña nueva empieza con la bodega vacía
6	Guarda un nombre, haz clic derecho sobre la pestaña y elige **Duplicar**. En la copia, presiona **Borrar el nombre** y luego recarga la pestaña original	La pestaña original **sigue** mostrando el nombre	Al duplicar, el navegador copia la bodega, pero cada pestaña queda con la suya

Prueba extra en la consola. En la pestaña **Console** de F12, escribe una línea a la vez:

```
sessionStorage.setItem("nombre", "Luis")
sessionStorage.getItem("nombre")      // "Luis"
sessionStorage.getItem("edad")        // null: esa clave no existe
```

Después recarga la página 2: ahora saluda a Luis, aunque nunca usaste la página 1. Esto demuestra que cualquiera puede cambiar lo que hay en la bodega, y por eso sessionStorage no sirve para guardar información secreta.

Preguntas, retos y errores comunes

Preguntas de reflexión. Respóndelas con lo que observaste en las pruebas:

1. ¿Por qué la página 2 conoce tu nombre si nunca vio la caja de texto de la página 1?
2. ¿Qué devuelve `getItem` cuando no hay nada guardado, y qué pasaría si el código no lo revisara con el `if`?
3. ¿En qué se diferencian recargar la página y cerrar la pestaña?
4. Después de la prueba extra en la consola, ¿guardarías una contraseña en sessionStorage? ¿Por qué?

Retos

- ☐ **Reto 1:** agrega en la página 1 una segunda caja para la ciudad y guárdala con la clave `ciudad`. Haz que la página 2 diga *Hola, Ana, de Bogotá*.
- ☐ **Reto 2:** en la página 1, si la caja está vacía, no guardes nada y muestra un mensaje pidiendo el nombre.
- ☐ **Reto 3:** cambia `sessionStorage` por `localStorage` en los dos archivos JS. Guarda un nombre, cierra el navegador por completo y vuelve a abrir `pagina2.html`. ¿Qué cambió?
- ☐ **Reto 4:** agrega en la página 2 un botón que borre **todo** con `sessionStorage.clear()`.

Errores comunes

Lo que ves	Por qué pasa	Solución
Cannot read properties of null (reading 'addEventListener')	Un `id` del JS no coincide con el del HTML, o falta `defer` en el `<script>`	Compara `btn-guardar`, `btn-borrar`, `nombre` y `saludo`, y revisa que el `<script>` tenga `defer`
La página 2 siempre dice *No hay ningún nombre guardado*	La clave está escrita distinto en los dos archivos, por ejemplo `"nombre"` y `"Nombre"`	Las claves distinguen mayúsculas: escríbela igual en `setItem` y `getItem`
Al hacer clic no pasa nada y la consola muestra *404*	La ruta del `<script>` no coincide con la carpeta	Revisa que la carpeta se llame `JS` y el archivo exista dentro
El nombre aparece, pero la página recarga sin ir a la página 2	La ruta de `window.location.href` está mal escrita	Debe ser exactamente `"pagina2.html"`
No funciona al abrir con doble clic	Algunos navegadores, como Firefox, son más estrictos con los archivos locales	Usa Chrome o Edge, o abre el proyecto con la extensión Live Server de VS Code

Lo que aprendiste: `setItem` guarda, `getItem` lee y `removeItem` borra. El dato sobrevive al cambio de página y a la recarga, pero desaparece al cerrar la pestaña, y cada pestaña tiene su propia bodega. Esta misma idea es la base del inicio de sesión del portafolio.